

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	R. Anshu Abhishek	Reg.No.:	192311034
Department:		Date:	14-08-26

BLEM

to solve
blems. (BL3)

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

on.
I
overflow

adder
als are
addition

3
In a
signed

4. multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I, **D.Ahaen Ab hek** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION						
Problem Understanding (20%)					Solution	Process(25%) 3
						Accuracy of Calculation (20%)
						Interpretation of Results(15%)
Q1	Q2	Q3	Q4	Q5	Method Selection(20%)	
2						

CALCULATION:						
Level 3						
Level 2						
Level 1						

RUBRICS FOR						
Criteria					method/formula with strong justification(4)	
Problem Understanding	Accuracy of Calculation	25%	Clearly interprets problem, identifies all parameters and		Logical, systematic steps with correct approach throughout(5)	
	Interpretation of Results	20%	Weightage 20%		All calculations accurate; correct final answer(4)	
Method Selection		15%	Level 4 - Exemplary		Clearly interprets results with units and	
Solution Process	20%				requirements accurately(4)	
					Selects most appropriate	

relevance(3)	elements(2)	
Proficient	Inappropriate or partially correct method(2)	No clear process(0)
Understands problem with minor gaps(3)	Disorganized steps; multiple errors(3)	No valid calculations(0)
Selects Correct method with minor erTOrs(3)	Frequent errors; incorrect final answer(2)	No interpretation(0)
Mostly correct steps with minor mistakes(4)	Limited or unclear interpretation(1)	
Minor calculation errors(3)		
Basic interpretation with minor omissions(2)	Beginning	
Developing	Misinterprets or unable to understand problem(1)	
Partial understanding; misses key	Unable to select method(1)	

SIMATS ENGINEERING — Department of Computer Science & Engineering

Assessment Tool 1 — Analytical Problem Solving | CSA1223 Computer Architecture | CO2

Assessment Tool 1 — Analytical Problem Solving

Course Code: CSA1223 **Course Title:** Computer Architecture **CO Assessed:** CO2 **Weightage:** 45% **Total Marks:** 100

Question 1

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Solution

Given data: Operand X = +45, Operand Y = -23, register width = 8 bits, valid signed range = -128 to +127.

Step 1 — Binary conversion of the operands

+45 in pure binary (8-bit): $45 = 32 + 8 + 4 + 1$, so **+45 = 00101101**.

To represent -23, first write +23 in binary: $23 = 16 + 4 + 2 + 1 \rightarrow 00010111$. Invert every bit (1's complement) $\rightarrow 11101000$, then add 1 $\rightarrow$ **-23 = 11101001**.

Both operands now occupy the full 8-bit field with the sign carried in the MSB (bit 7), so no separate sign-alignment step is required — that is the advantage of 2's complement.

Step 2 — Addition: (+45) + (-23)

```
00101101 (+45)
+ 11101001 (-23)
-----
```

1 00010110

The 9th bit (the carry out of bit 7) is dropped because the register is only 8 bits wide, leaving **00010110**.

Converting back: $00010110 = 16 + 4 + 2 = \mathbf{22}$, and the sign bit is 0, so the result is **+22**.

Step 3 — Overflow check for the addition

The operands have opposite signs (+45 is positive, -23 is negative). Two operands of opposite sign can never overflow a 2's complement addition, because the true sum must lie strictly between the two operands and therefore always stays inside the representable range. **Overflow = No**.

Step 4 — Subtraction: (+45) - (-23)

Subtraction is implemented as “add the 2's complement of the subtrahend,” so:

$$(+45) - (-23) = (+45) + (2\text{'s complement of } -23) = (+45) + (+23)$$

```
00101101 (+45)
+ 00010111 (+23)
-----
01000100
```

$01000100 = 64 + 4 = \mathbf{68}$, sign bit 0 → result is **+68**.

Step 5 — Overflow check for the subtraction

This step added two positive operands (+45 and +23) and produced a positive result (sign bit = 0). Overflow after adding two same-signed numbers would show up only if the result flipped sign, which did not happen here. **Overflow = No**.

Step 6 — How the hardware actually flags overflow

A real ALU does not “think” about signs the way we just did — it uses the carry bits directly:

V (overflow flag) = $C_{in} \text{ XOR } C_{out}$, where C_{in} is the carry **into** the sign bit (bit 7) and C_{out} is the carry **out of** the sign bit.

For the addition above, C_{in} into bit 7 = 0 and C_{out} of bit 7 = 0, so $V = 0 \text{ XOR } 0 = 0$ → no overflow, agreeing with the sign-based check. The same rule flags overflow instantly in hardware without needing to compare operand signs first, which is why processors implement it this way rather than checking signs in software.

Final Answer

Operation	Binary	Result	Decimal	Overflow
$(+45) + (-23)$	00010110	+22	No	
$(+45) - (-23)$	01000100	+68	No	

Interpretation

Both results (+22 and +68) lie comfortably inside the 8-bit signed range of -128 to +127, so the processor's flags correctly report no overflow in either case. This example also shows why 2's complement is attractive for hardware designers: addition and subtraction use the *same* adder circuit, and the sign bit needs no special handling — it is simply the most significant bit of the result.

Question 2

A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate

and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Solution

Given data: A = 1101 (13), B = 0111 (7), 4-bit Carry Look-Ahead Adder, input carry C0 = 0.

Step 1 — Generate and Propagate signals

For each bit position i: Generate $G_i = A_i \cdot B_i$ (a carry is produced regardless of the incoming carry), and Propagate $P_i = A_i \oplus B_i$ (an incoming carry would be passed straight through). $\oplus$

i A_i B_i $G_i = A_i \cdot B_i$ $P_i = A_i \oplus B_i$ $\oplus$ 0 1 1 1 0
 1 0 1 0 1
 2 1 1 1 0
 3 1 0 0 1

Step 2 — Compute every carry in advance using the CLA equations

$C_1 = G_0 + P_0 \cdot C_0 = 1 + (0)(0) = 1$
 $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 0 + (1)(1) + (1)(0)(0) = 1$ $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
 $= 1 + (0)(0) + (0)(1)(1) + (0)(1)(0)(0) = 1$ $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$
 $= 0 + (1)(1) + (1)(0)(0) + (1)(0)(1)(1) + (1)(0)(1)(0)(0) = 1$

Notice that every C_i is written purely in terms of the *original* G's and P's, not in terms of the previous carry chain — that is what lets all four carries be evaluated simultaneously by parallel AND/OR gates instead of waiting for a ripple.

Step 3 — Sum bits

$S_i = P_i \oplus C_i \rightarrow S_0 = 0 \oplus 0 = 0, S_1 = 1 \oplus 1 = 0, S_2 = 0 \oplus 1 = 1, S_3 = 1 \oplus 1 = 0$. So Sum = 0100, and $C_4 = 1 \oplus \oplus \oplus \oplus$ gives the 5-bit result **10100**.

Verification: 1101 (13) + 0111 (7) = 10100 (20). ✓

Step 4 — Delay comparison

In a ripple carry adder, each full adder's carry-out is not available until the carry-in from the previous stage has settled, so the carry has to physically ripple across all n stages. Worst-case delay grows linearly: $T(RCA) \approx n \times t_{\text{carry}}$ (about 2 gate delays per stage), which for a 4-bit adder is roughly 8 gate delays, and the delay keeps growing as bit-width increases (16-bit, 32-bit, etc.).

In the CLA, every C_i is produced from a two-level AND-OR expression built directly from $G_0..G_i$ and $P_0..P_i$, so the carry into any stage is available after a fixed number of gate delays (about 4, regardless of n) — the stages do not have to wait on each other.

Adder type	Approx. delay for n bits	Growth with n
Ripple Carry Adder	$\approx n \times t_{\text{carry}}$	Linear,
Carry Look-Ahead Adder	$\approx \text{constant}$ (≈ 4 gate delays)	Independent of n, $O(1)$ -ish for the carry network

Final Answer

Sum = **10100 (decimal 20)**, with $S_3 S_2 S_1 S_0 = 0100$ and final carry $C_4 = 1$.

Interpretation

Because the CLA computes generate/propagate signals for all bit positions in parallel and derives every carry directly from them, it removes the sequential carry-ripple bottleneck. This is precisely why high-performance processors use carry look-ahead (or hybrid look-ahead/block-carry-lookahead) adders in their ALUs — the speed gain becomes even more significant as the operand width grows from 4 bits to 32 or 64 bits, where a ripple design would otherwise dominate the critical path of every arithmetic instruction.

Question 3

In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Solution

Given data: Multiplicand $M = -6$, Multiplier $Q = +5$. Since the result can require more bits than either operand, both operands are represented in 5-bit 2's complement (range -16 to $+15$) so the running product stays inside the accumulator.

Step 1 — Binary representation

$+6 = 00110 \rightarrow \text{invert} \rightarrow 11001 \rightarrow +1 \rightarrow \mathbf{M = 11010 (-6)}$. $\mathbf{Q = 00101 (+5)}$.

Step 2 — Roles of the registers

- **A (Accumulator):** holds the running partial product, initialised to 0.
- **Q (Multiplier register):** initially holds the multiplier; after each shift it gradually fills with the low-order bits of the final product.
- **Q (Booth bit):** -1 an extra flip-flop, initially 0, that remembers the bit that has just shifted out of Q. It is compared with Q_0 to decide the operation for that cycle.

Booth's rule, applied to the pair (Q_0, Q_{-1}) : 00 or $11 \rightarrow$ no arithmetic operation (just shift); $10 \rightarrow -1 A = A - M$; $01 \rightarrow A = A + M$. After every cycle, the combined register (A, Q, Q_{-1}) is shifted right -1 arithmetically (sign bit of A is preserved) by one position.

Step 3 — Iteration table

				Shift result $\rightarrow$
Iter	Q_0	Q_{-1}	Operation	A after op
1	1	0	$A = A - M$	00011, 00010, 1
2	0	1	$A = A + M$	11101 11110, 10001, 0
3	1	0	$A = A - M$	00100 00010, 01000, 1
4	0	1	$A = A + M$	11100 11110, 00100, 0
5	0	0	no operation	— 11111, 00010, 0

(5 iterations because the operands are 5-bit; each row's "operation" is applied before that iteration's arithmetic shift.)

Step 4 — Final product

After the 5th shift, $A = 11111$ and $Q = 00010$. Concatenating them gives the 10-bit product $\mathbf{A:Q = 11\ 1110\ 0010}$, i.e. 1111100010. Since the sign bit is 1, take the 2's complement to find the magnitude: invert $\rightarrow 0000011101$, add 1 $\rightarrow 0000011110 = 30$, so the value is **-30** .

Verification: $(-6) \times (+5) = -30$. ✓

Final Answer

Product = **1111100010** = **-30 (decimal)**₂.

Interpretation

Booth's algorithm never needs to know the signs of the operands separately — the same A-M / A+M / no-op rule works uniformly for any combination of positive and negative operands, because the operands are already stored in 2's complement. In this example, two of the five cycles required no arithmetic at all (rows 1's "no operation" cases would occur more often for multipliers with long runs of 0s or 1s), which is exactly how Booth's algorithm reduces the number of add/subtract operations compared to a naive shift-and-add multiplier that would perform one addition for every '1' bit in the multiplier regardless of pattern.

Question 4

A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Solution

Given data: Dividend = 13 (1101), Divisor = 3 (0011), 4-bit unsigned division, so a 5-bit remainder register A is used to safely detect sign after subtraction.

4(a) Restoring Division

Initialise A = 00000, Q = 1101, M = 00011.

Cycle	Shift A,Q left A - M Sign Action A after cycle	Q after cycle	
	00011-000 11=11110		00001-000 11=11110
		1 A=00001, Q =1010	Neg restore, Q0=0
	00001 1010		
	00011-000 11=00000		
2 A=00011, Q =0100	Pos keep, Q0=1 00000 0101		
	00000-00 011=11101	Neg restore, Q0=0	0100
3 A=00000, Q =1010	00001-000 11=11110	00000 1010 00001	
4 A=00001, Q =0100	Neg restore, Q0=0		

Whenever the trial subtraction goes negative, the remainder register is *restored* by adding the divisor back — this happened in cycles 1, 3 and 4, i.e. 3 out of 4 cycles needed an extra "undo" addition on top of the subtraction.

Result: Quotient Q = 0100 = **4**, Remainder A = 00001 = **1**.

4(b) Non-Restoring Division

Same operands. Rule: if the previous remainder was non-negative, shift and subtract M; if it was negative, shift and add M instead (no restore step is ever performed mid-way).

Cycle Operation A after op Sign Q0

1 shift, A-M: 00001-00011 1101, +M

2 (prev neg) shift, A+M: 11110 Neg 0 00000 Pos 1
11101-shift-1

3 (prev pos) 11101 Neg 0

	Cycle Operation A after op Sign Q0 shift, A-M
4 (prev neg) shift, A+M	11110 Neg 0

Because the final remainder (11110) is negative, a **single** correction step is applied at the very end: $A = A + M = 11110 + 00011 = 00001$.

Result: Quotient $Q = 0100 = 4$, Remainder $A = 00001 = 1$ — identical to the restoring method.

Step — Verification: $3 \times 4 = 12$, and $13 - 12 = 1$, so quotient 4 and remainder 1 is correct for both methods.

Final Answer

$13 \div 3 = \text{Quotient} = 4, \text{Remainder} = 1$ (confirmed by both algorithms).

Comparison and Interpretation

Aspect	Restoring Division	Non-Restoring Division
Extra “restore” additions	Needed every time a trial subtraction is negative (3 in this example)	None — a wrong guess is simply corrected on the <i>next</i> cycle by adding instead of subtracting
Total arithmetic operations for $n = 4$	Up to $2n$ in the worst case (subtract + restore)	Exactly n , plus at most one final correction
Control complexity	Simpler decision logic, but wastes cycles undoing work	modern CPUs Slightly more complex sign bookkeeping, but never discards useful work Preferred, and extended further into SRT/Radix-4 division
Typical hardware use	Rarely used alone in	

Since restoring division repeatedly performs a subtraction and then throws it away by adding the divisor back, it wastes clock cycles whenever the trial quotient bit is 0. Non-restoring division removes this waste by folding the “undo” into the very next iteration’s operation choice, so on average it completes in roughly half the arithmetic operations for the same number of bits — this is exactly why real processors implement non-restoring division (or its faster derivatives) rather than the restoring version.

Question 5

A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Solution

Given data: Decimal number = -13.25.

Step 1 — Convert the magnitude to binary

$13 = 1101$. $0.25 = 0.01$ (since $0.25 = 1 \times 2^{-2}$). So $13.25 = {}_2 2^{-} 1101.01_2$.

Step 2 — Normalize

$1101.01 = 1.10101 \times 2^3$ (moved the binary point 3 places left, so the true exponent is 3).

Step 3 — Single precision (1 sign + 8 exponent + 23 mantissa bits)

Two practical issues surface in this kind of addition: **normalization** requires shifting the smaller-exponent mantissa right before the operation, which can push its low-order bits beyond the fixed mantissa width — this is a source of **precision loss**, since bits shifted past the last mantissa position are dropped or merged during **rounding**. In this particular example the exponent difference was only 1, so no bits were lost; but the same procedure applied to operands with a much larger exponent gap (a very large number added to a very tiny one) can shift the smaller operand's significant bits out of range entirely, silently discarding its contribution — a limitation inherent to any fixed-width floating-point format.